

W4D4 - Progetto di fine modulo M1

Come progetto di fine modulo, andremo ad applicare tutte le conoscenze acquisite nelle ultime 4 settimane. L'esercizio consiste nel simulare con **inetsim** un **server web** che risponda a richieste **http** e **https**, simuleremo anche un **server DNS** che dovrà servire l'indirizzo IP della kali in risposta al nome di dominio **epicode.internal**.

Per interrogare il server web utilizzeremo il **browser chrome** installato **sulla macchina windows**.

Andremo quindi a metterci in ascolto tramite **wireshark** e intercetteremo i pacchetti per studiarli. Dovremo evidenziare eventuali differenze tra le comunicazioni con il protocollo http e https.

Configurazione InetSim

La configurazione di inetsim avviene attraverso il file di configurazione `/etc/inetsim/inetsim.conf`, andremo a modificare l'indirizzo IP su cui il programma si metterà in ascolto e anche le impostazioni relative al servizio DNS per inserire un record per l'indirizzo **epicode.internal**.

La prima modifica da implementare è valorizzare l'impostazione

`service_bind_address` in modo da

mettere il programma in ascolto

sull'interfaccia legata all'indirizzo IP della nostra Kali, in questo esempio,

`192.168.50.100`. In questo modo inetsim

sarà in grado di ricevere connessioni in

entrata da altre macchine appartenenti

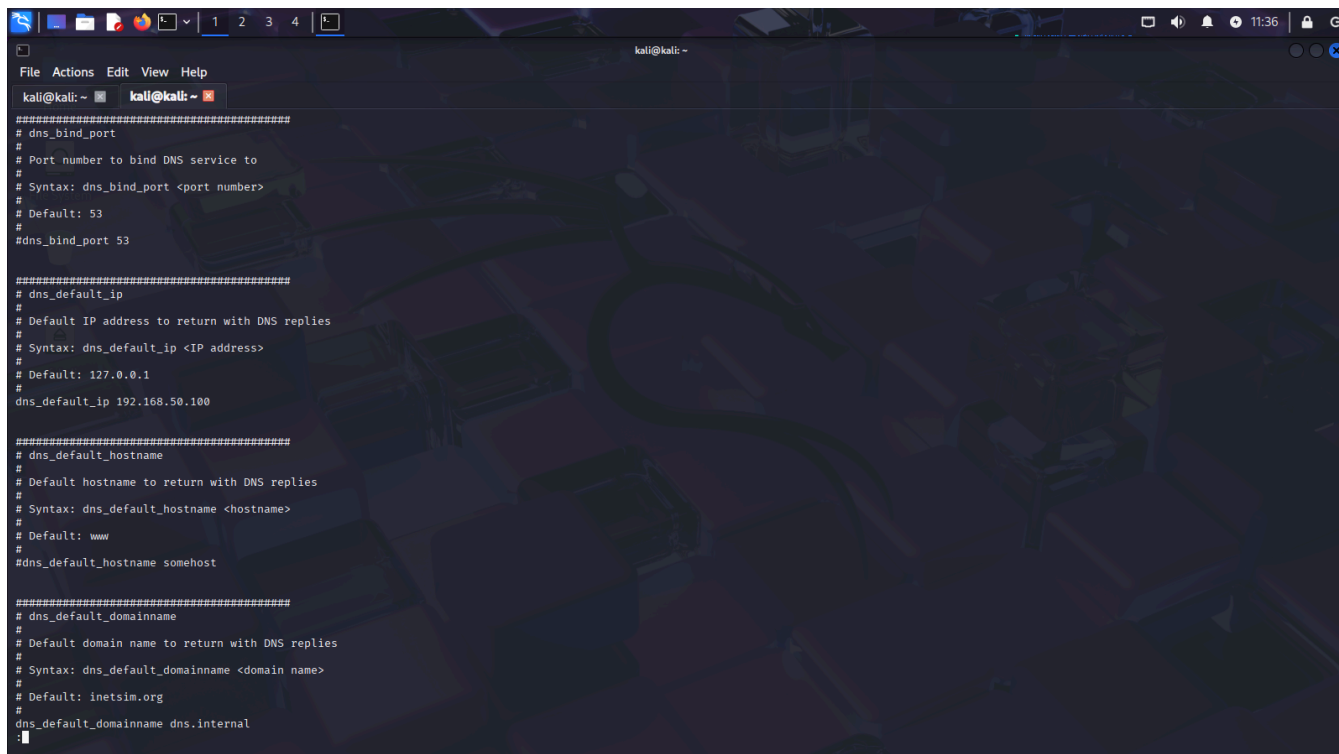
alla stessa sottorete. Di default è

impostato il valore `127.0.0.1` che è legato all'interfaccia di loopback, permettendo ad

inetsim di ascoltare solo connessioni provenienti dalla stessa macchina.

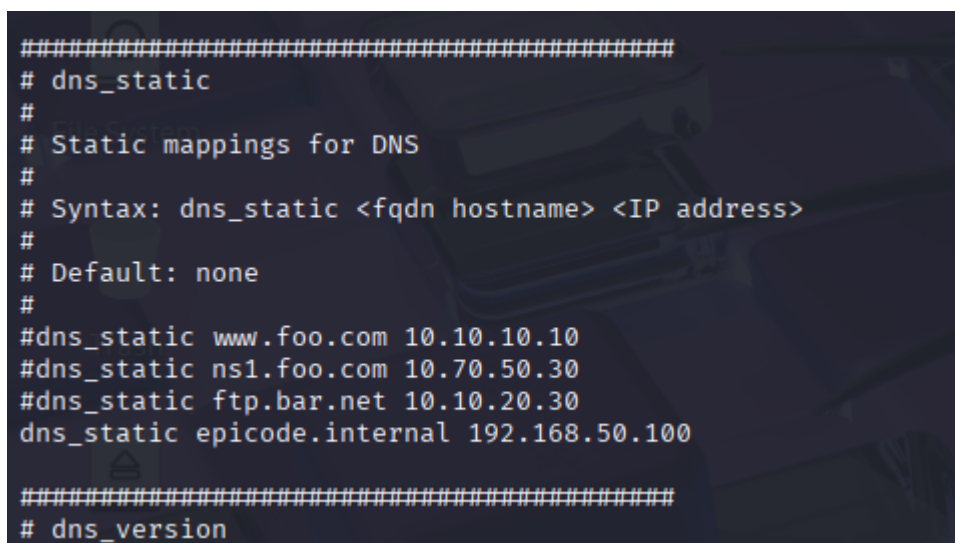
```
#####
# service_bind_address
#
# IP address to bind services to
#
# Syntax: service_bind_address <IP address>
#
# Default: 127.0.0.1
#
#service_bind_address 10.10.10.1
service_bind_address 192.168.50.100
#####
```

Per configurare il servizio DNS dovremo scendere più in basso nel file fino a trovare le voci che iniziano con `dns_`



```
File Actions Edit View Help
kali@kali: ~ kali@kali: ~
#####
# dns_bind_port
#
# Port number to bind DNS service to
# Syntax: dns_bind_port <port number>
# Default: 53
# dns_bind_port 53
#####
# dns_default_ip
#
# Default IP address to return with DNS replies
# Syntax: dns_default_ip <IP address>
# Default: 127.0.0.1
# dns_default_ip 192.168.50.100
#####
# dns_default_hostname
#
# Default hostname to return with DNS replies
# Syntax: dns_default_hostname <hostname>
# Default: www
# dns_default_hostname somehost
#####
# dns_default_domainname
#
# Default domain name to return with DNS replies
# Syntax: dns_default_domainname <domain name>
# Default: inetsim.org
# dns_default_domainname dns.internal
:
```

Andremo a modificare il valore di `dns_default_ip` con lo stesso indirizzo IP impostato prima, e poi per ultimo, aggiungeremo una voce `dns_static` in cui assoceremo all'indirizzo IP della kali, il nome di dominio **epicode.internal**, come da screenshot seguente:



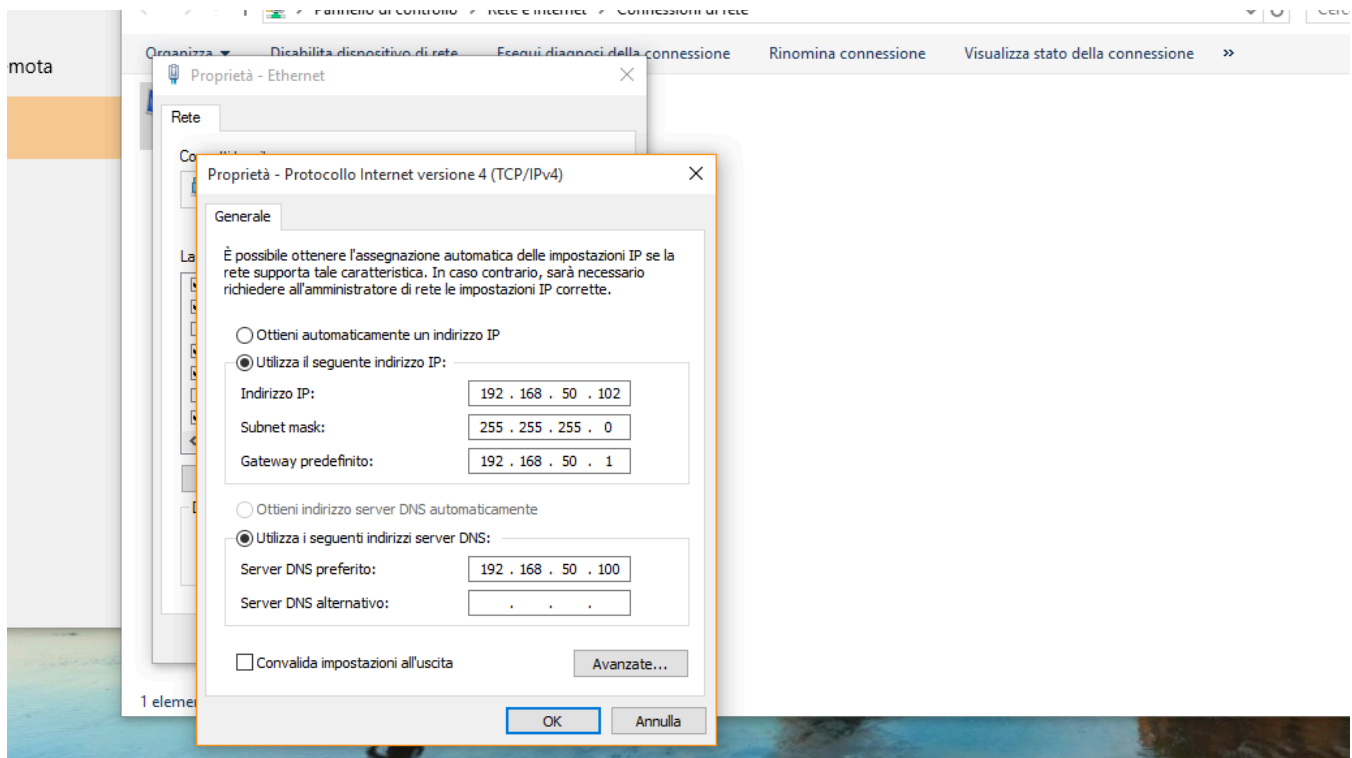
```
#####
# dns_static
#
# Static mappings for DNS
#
# Syntax: dns_static <fqdn hostname> <IP address>
#
# Default: none
#
#dns_static www.foo.com 10.10.10.10
#dns_static ns1.foo.com 10.70.50.30
#dns_static ftp.bar.net 10.10.20.30
dns_static epicode.internal 192.168.50.100
#####
# dns_version
```

Una volta salvato il file potremo avviare InetSim, il quale avvierà tutti i servizi e li metterà in ascolto sull'interfaccia legata all'IP che abbiamo impostato.

Configurazione Windows

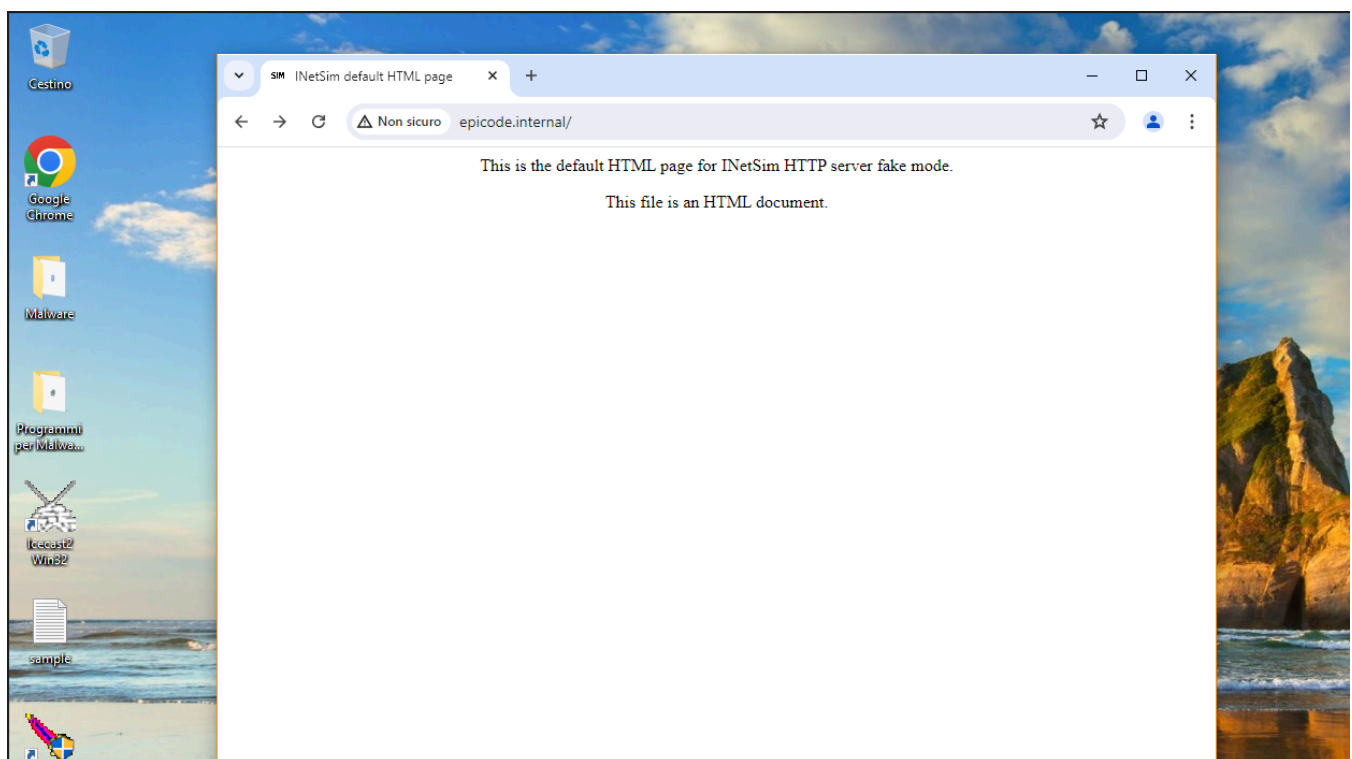
Adesso dovremo recarci sulla macchina Windows e modificare leggermente la configurazione della scheda di rete, in modo da utilizzare il server DNS che abbiamo appena configurato.

Cliccando sull'icona di rete in basso nella barra di Windows, apriremo le impostazioni della scheda di rete e poi andremo a configurare il protocollo **TCP/IPv4**, cambiando l'indirizzo IP del server DNS predefinito con quello della nostra macchina Kali. Si veda la figura a lato.

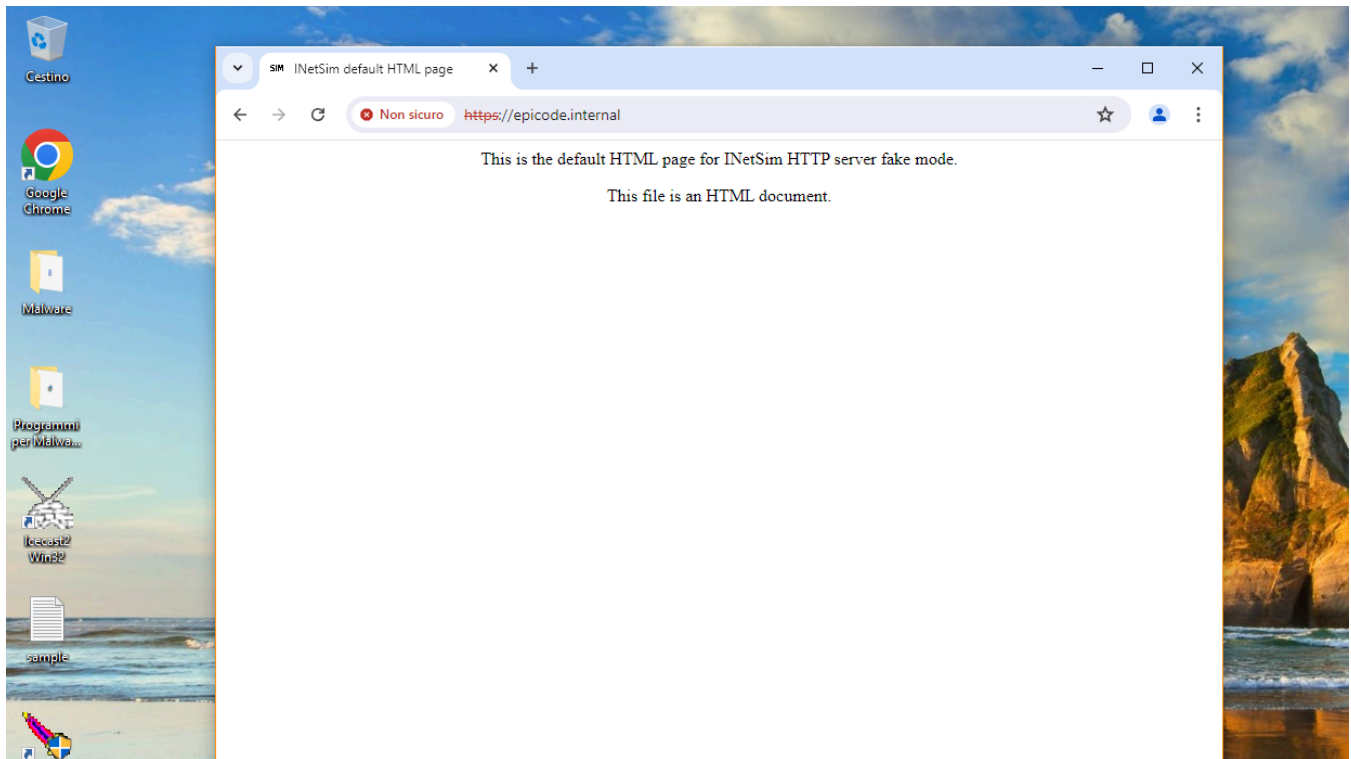


La configurazione è subito attiva, quindi possiamo testarne il funzionamento aprendo un browser e puntandolo all'indirizzo <http://epicode.internal>.

Se tutto è andato per il verso giusto, dovremmo vedere la pagina iniziale di Inetsim.



Uguualmente, se proviamo a richiedere il protocollo https, dovremmo ricevere un avvertimento da parte di chrome riguardo all'affidabilità del certificato del sito, ma una volta scelto di continuare ugualmente, dovremmo vedere la stessa pagina, protetta da cifratura.

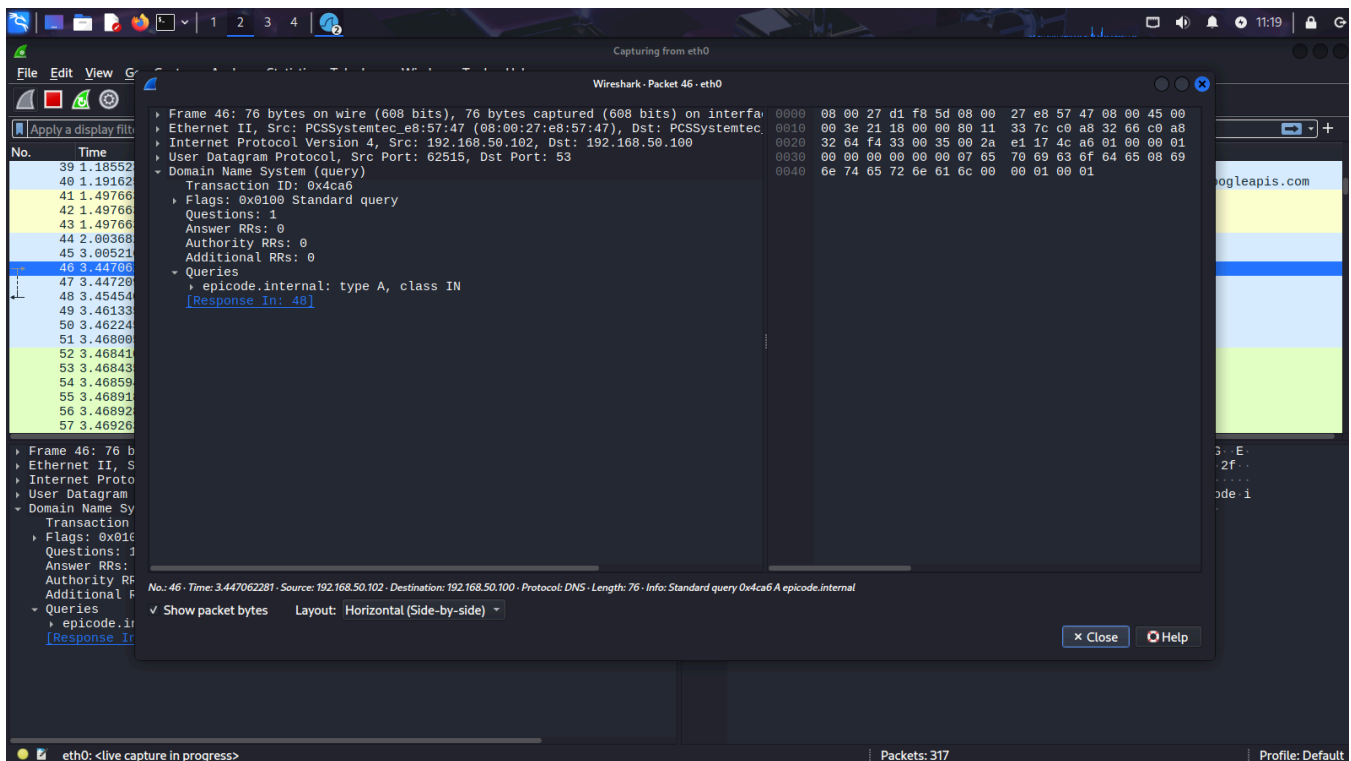


Analisi pacchetti con Wireshark

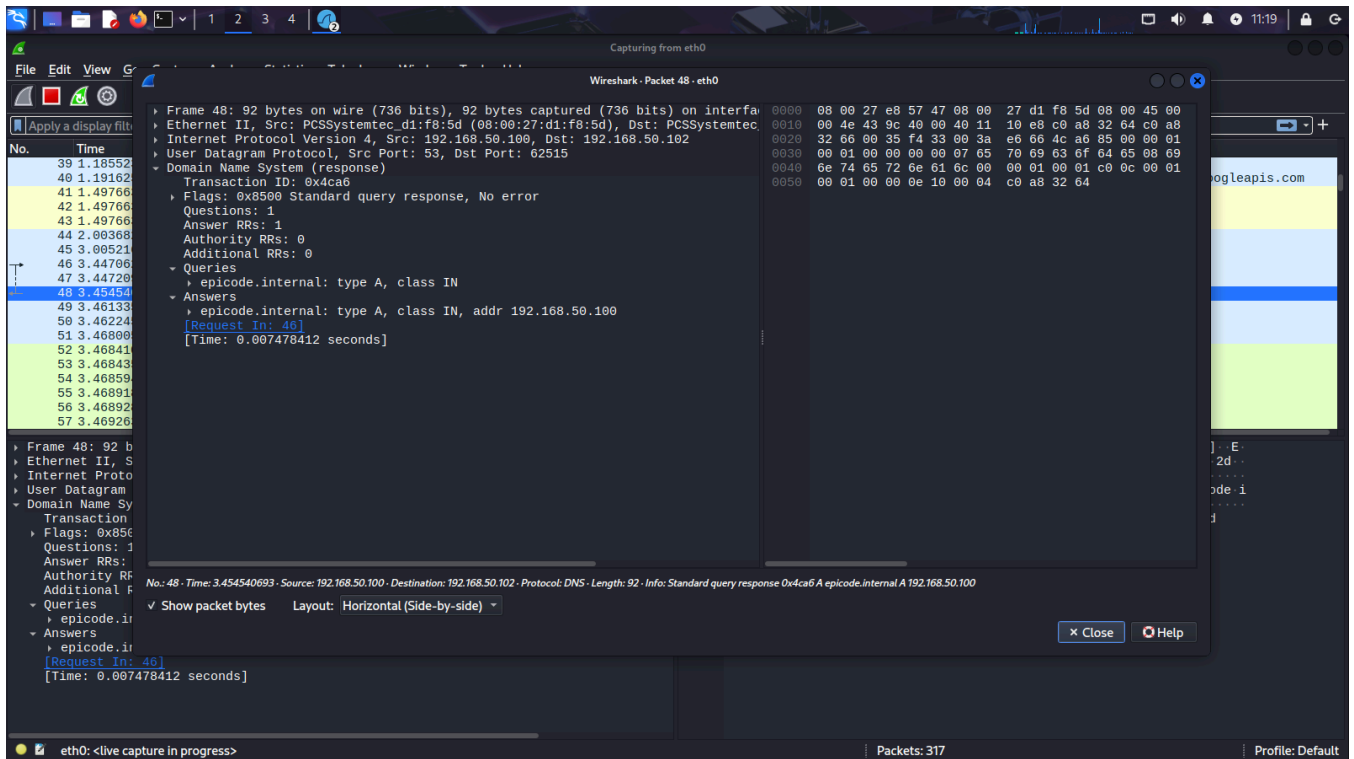
Tornati sulla nostra Kali, possiamo andare a vedere con Wireshark, quali pacchetti si sono scambiati i due sistemi.

DNS query

Il primo pacchetto interessante è quello con la richiesta del record DNS per l'indirizzo epicode.internal



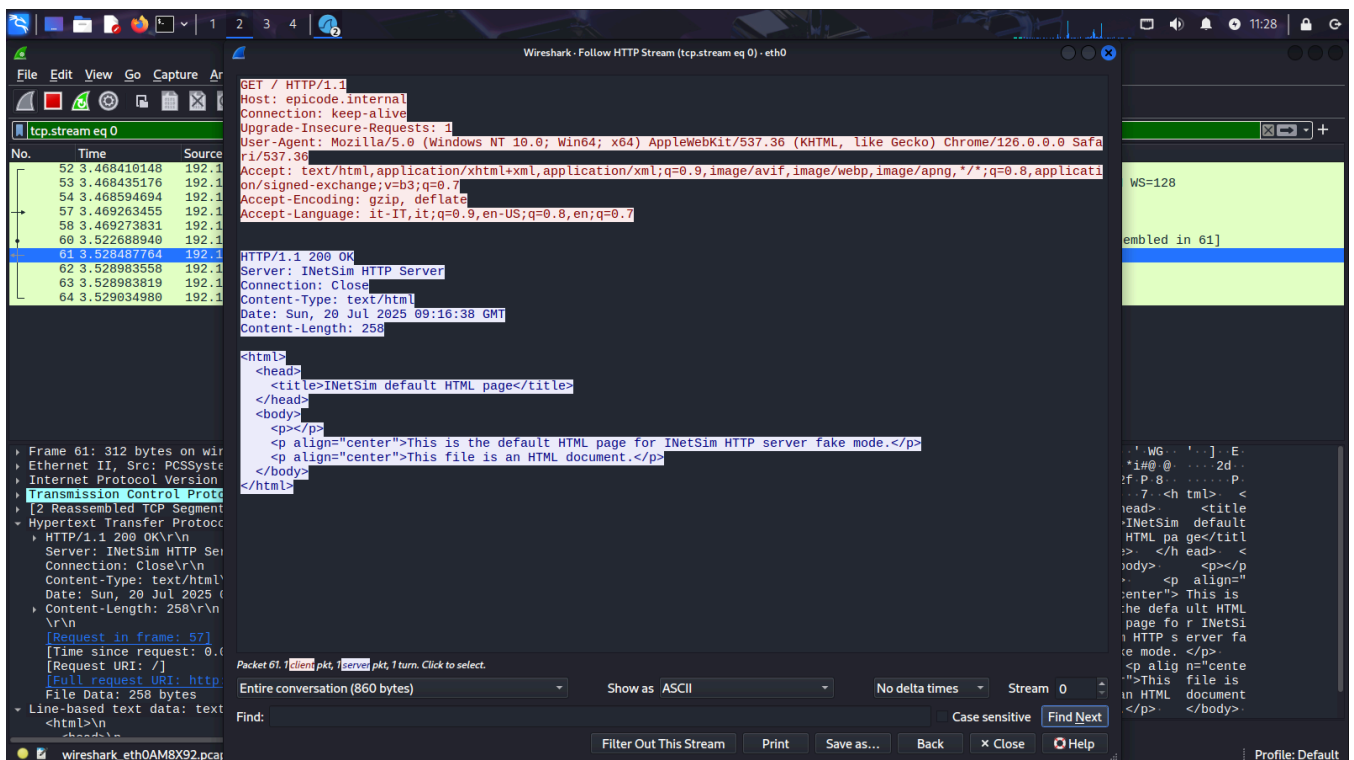
e la relativa risposta da parte del nostro server, con l'indirizzo IP della Kali.



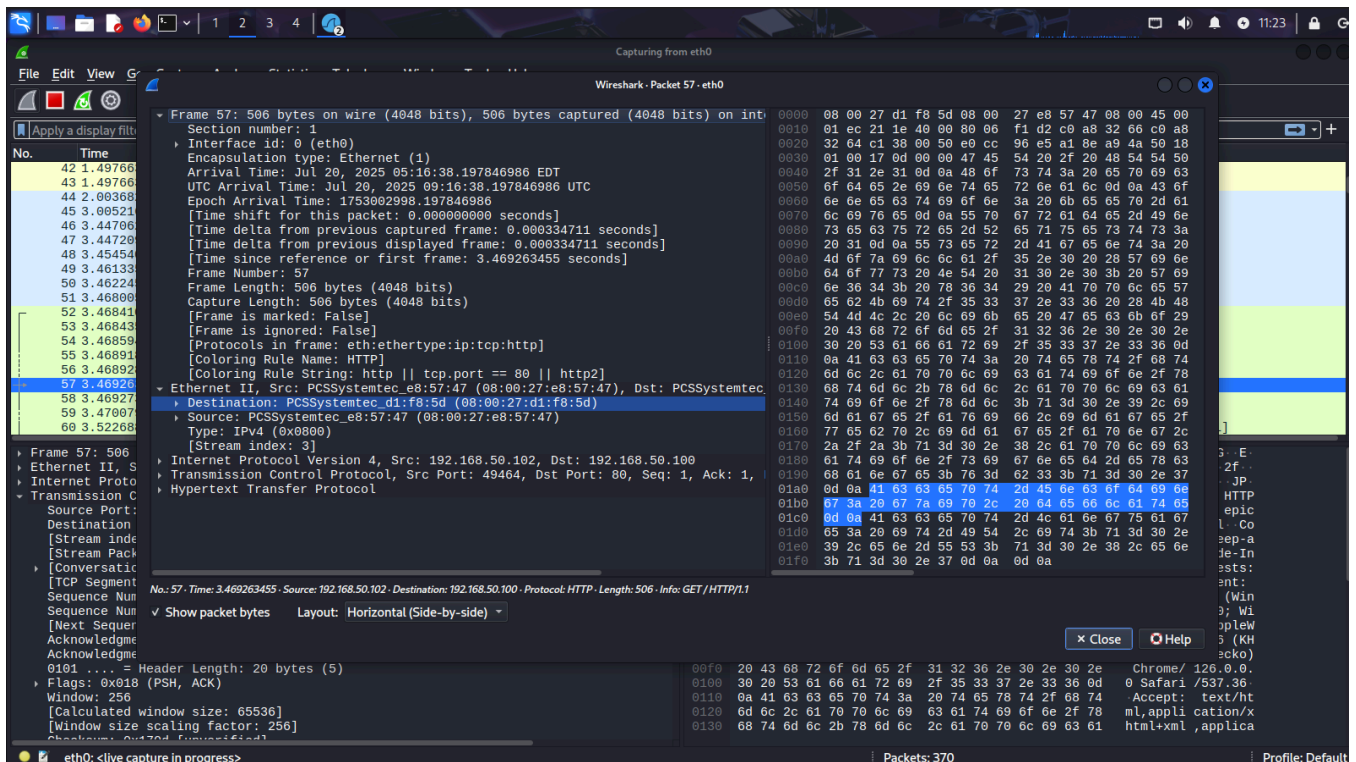
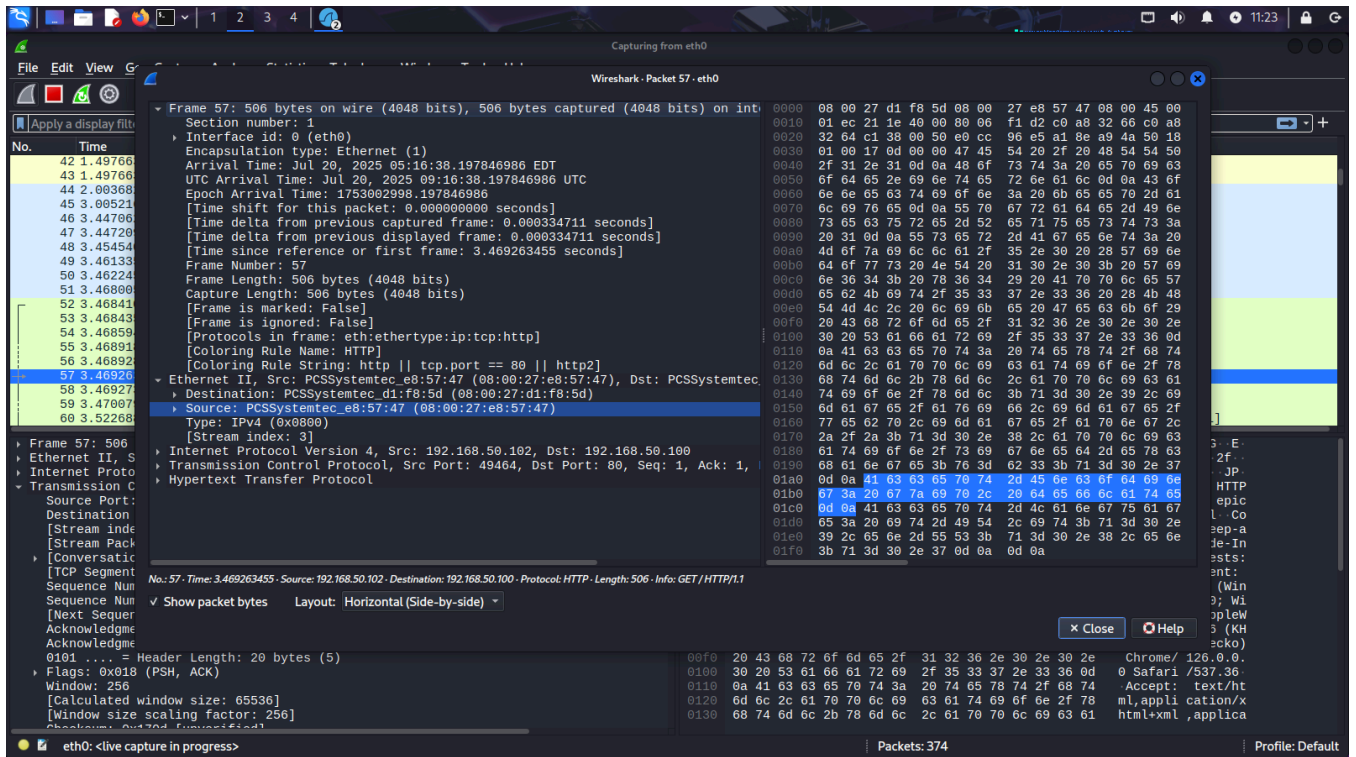
Adesso andiamo a verificare quale pagina è stata visualizzata dal browser andando ad analizzare i pacchetti http (in chiaro) e https (cifrati).

HTTP query

Vediamo che nella richiesta in chiaro sono subito visibili tutte le informazioni che il server e il client si sono scambiati.

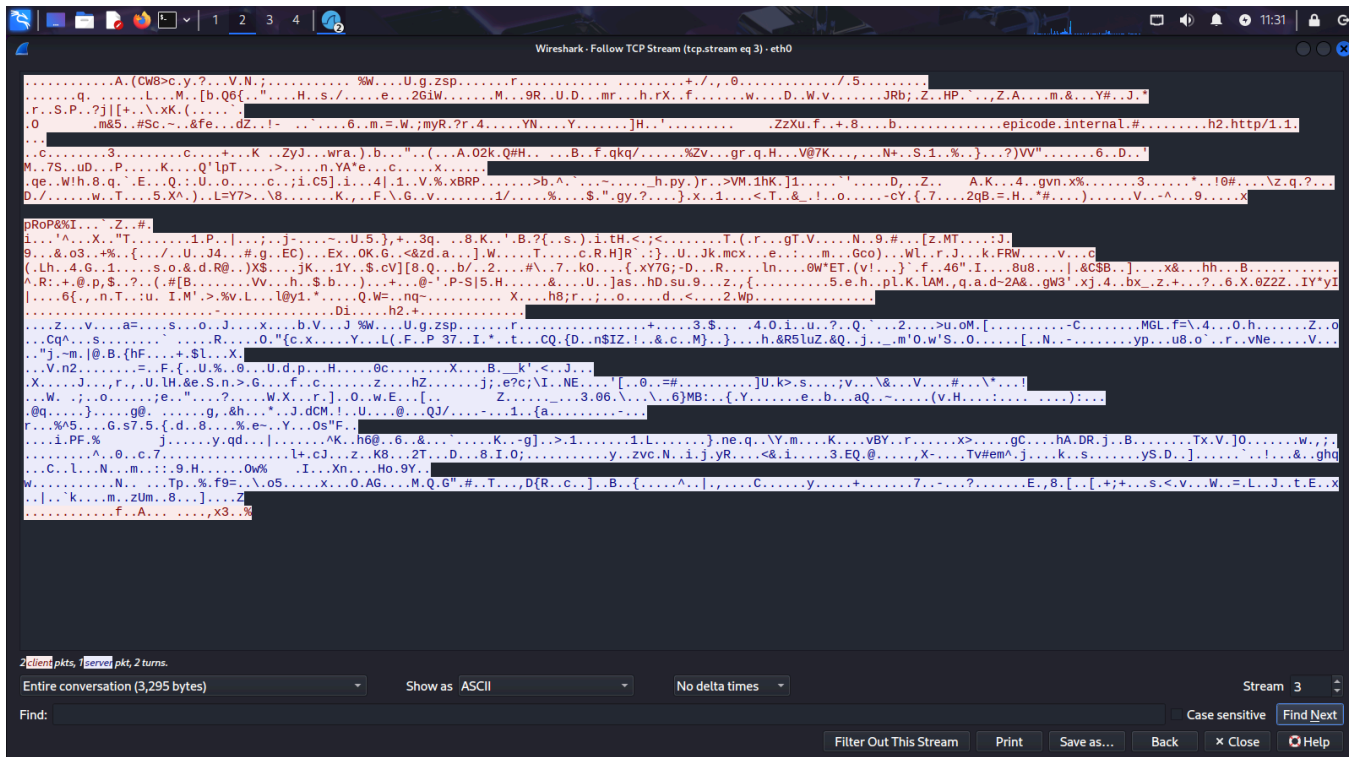


e riusciamo a vedere nei vari strati di incapsulamento, a livello ethernet, i MAC address delle due macchine, SOURCE e DESTINATION.

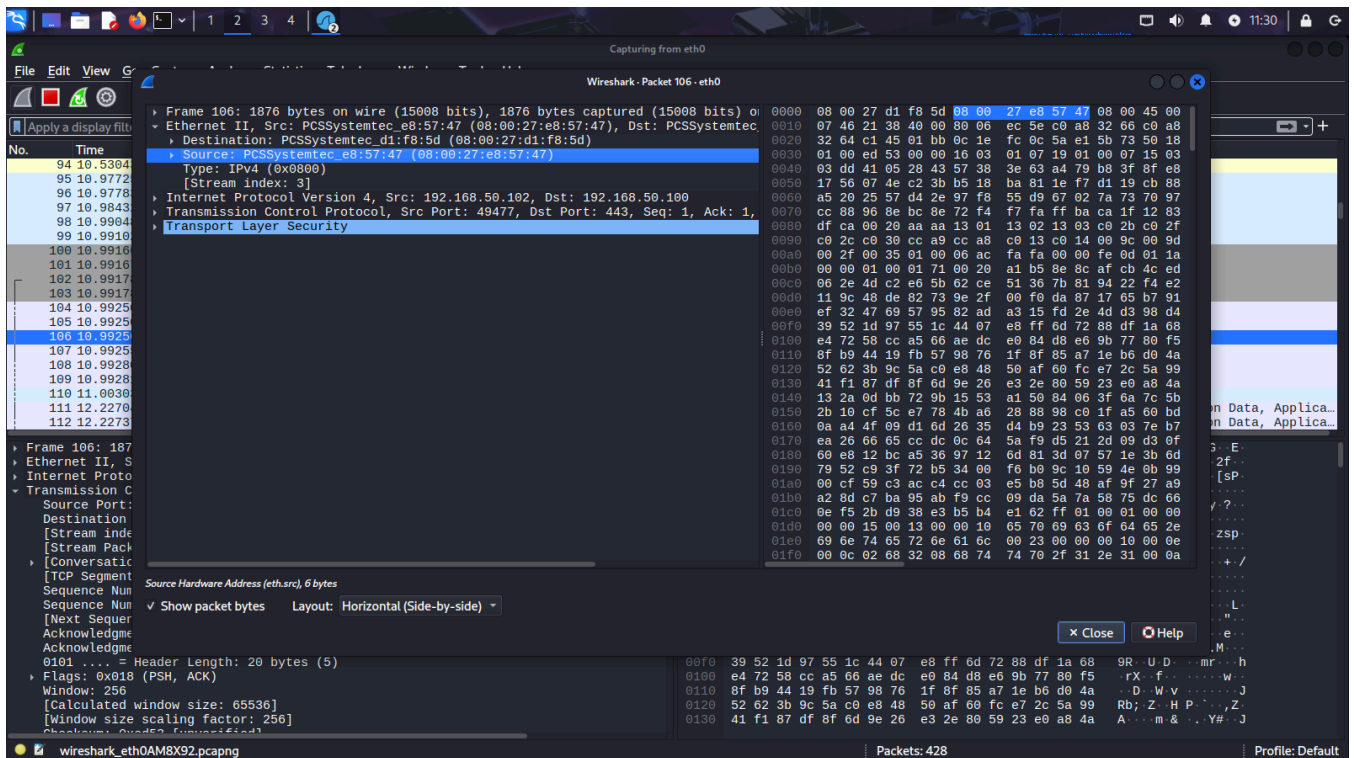


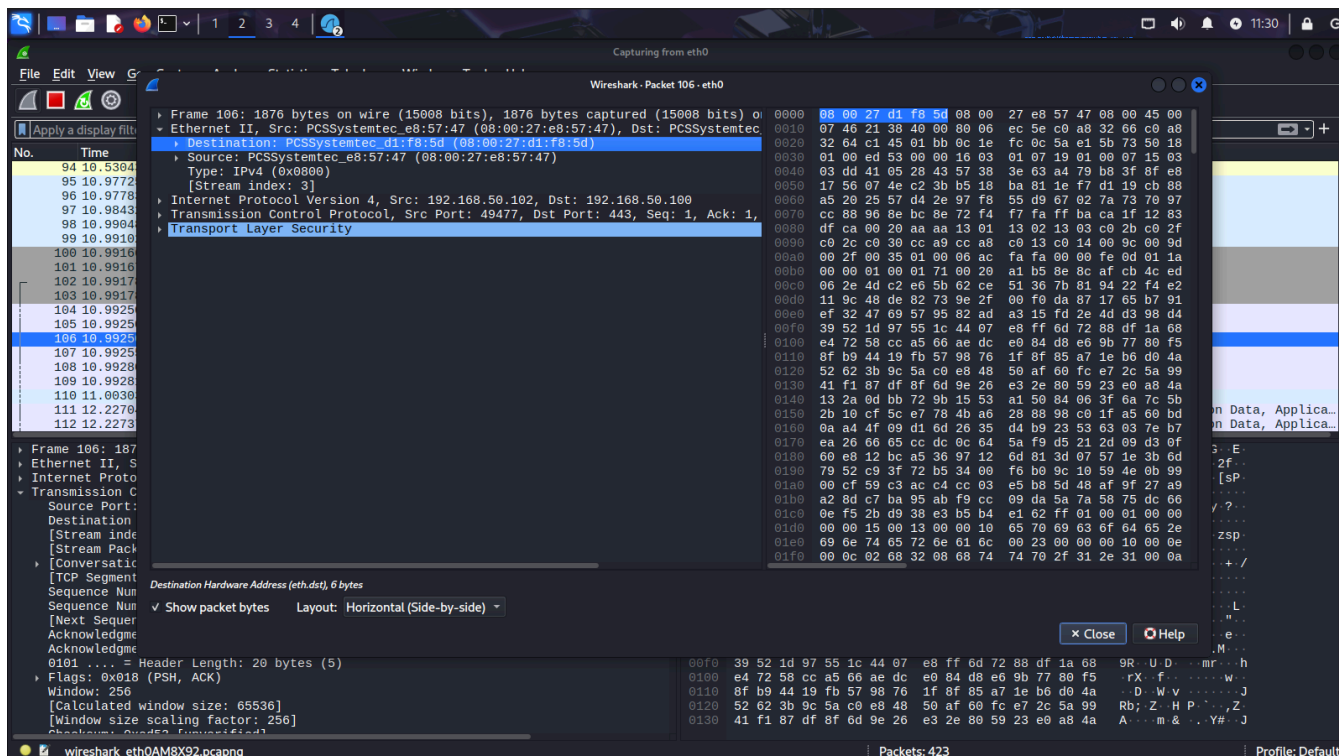
HTTPS query

Discorso diverso invece, per la richiesta fatta con protocollo https, in quanto i dati sono transitati tra le due macchine in maniera cifrata, quindi da WireShark non riusciremo a vederne il contenuto.



Ma andando ad analizzare l'incapsulamento, vedremo che nel livello Ethernet sono comunque visibili i MAC address delle due macchine.





La cifratura avviene quindi per il payload effettivamente trasportato, un soggetto attaccante potrà vedere alcune informazioni riguardo alla nostra comunicazione, come l'indirizzo IP e MAC del sorgente e del destinatario del pacchetto, ma non potrà leggerne il contenuto. Questo è specialmente importante per comunicazioni sensibili, come i dati di pagamento su uno store online, o i dati di accesso ad un servizio.

Extra - stabilire una connessione cifrata

Quello che rende sicura una connessione cifrata tra due macchine, è l'instauramento di una catena di fiducia (trust chain), le due macchine, prima di iniziare lo scambio di pacchetti contenenti dati sensibili, eseguono quello che si chiama 3-way handshake e che è alla base del protocollo TLS, in cui stabiliscono le basi per la cifratura delle comunicazioni successive.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.50.102	192.168.50.100	TCP	66	49515 → 443 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.000032761	192.168.50.100	192.168.50.102	TCP	66	443 → 49515 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
3	0.000260614	192.168.50.102	192.168.50.100	TCP	60	49515 → 443 [ACK] Seq=1 Ack=1 Win=65536 Len=0

Quello che si vede nell'immagine è uno dei vari passaggi in cui le due macchine, client e server, si scambiano informazioni sotto forma di 3-way handshake. La sequenza è sempre la stessa, il client invia un primo pacchetto `SYN`, in cui, in questo caso, comunica gli algoritmi supportati, la versione di TLS supportata e una stringa casuale "client", il server risponde con un pacchetto `SYN, ACK`, in cui comunica il proprio certificato SSL, i propri algoritmi di cifratura supportati, la versione di TLS e un'altra stringa casuale "server". L'ultimo pacchetto `ACK`, lo invia quindi il client dopo aver verificato il certificato con l'autorità che lo ha emesso, con questa verifica il client sarà sicuro di stare parlando con il server che ha richiesto originariamente.

Da qui in poi ci saranno altre sequenze di 3-way handshake (in base alla versione di TLS utilizzata) per stabilire la cifratura da utilizzare e per scambiarsi delle stringhe casuali crittografate in modo da verificare che la cifratura funzioni per entrambe le macchine.

In questo modo, una volta che il server e il client si saranno allineati sui metodi della crittografia da utilizzare, potranno iniziare a scambiarsi in sicurezza i dati sensibili senza preoccuparsi di eventuali soggetti in ascolto.

Con questo si conclude il progetto finale del primo modulo.

Ciao e buon hacking 😊

